

Using the Sequence-Space Jacobian to Solve and Estimate Heterogeneous-Agent Models

Chen Gao Bo Li

School of Economics, Peking University

April 20, 2026

Outline

1. Why Sequence Space?
2. Steady State
3. Computing HA Jacobians: the “fake news” algorithm
4. DAGs
5. Recap

Why Sequence Space?

Sequence Space vs. State Space

Motivation

- In heterogeneous-agent (HA) models, the economy's state is the **entire cross-sectional distribution** — an infinite-dimensional object.
- Sequence-space methods *sidestep* this challenge: they characterize **local dynamics around the steady state** without ever carrying the distribution as a state variable.
- The key insight: aggregate equilibrium dynamics can be represented in sequence space as mappings from **aggregate shock paths** to **aggregate outcomes**, with Jacobians arising as the local linear approximation of these mappings.

Sequence Space vs. State Space

State Space

Definition 1

A **state-space representation** describes the economy recursively. At date t , a state variable x_t summarizes all payoff-relevant history, and the model evolves as

$$x_{t+1} = f(x_t, \varepsilon_{t+1}), \quad y_t = g(x_t).$$

Given x_t , the future is independent of the past.

Sequence Space vs. State Space

Sequence Space

Definition 2

A **sequence-space representation** characterizes equilibrium as a mapping between entire time paths. We seek sequences $\{X_t\}_{t=0}^{\infty}$ such that equilibrium conditions hold at every date:

$$F(\{X_t\}_{t=0}^{\infty}, \{Z_t\}_{t=0}^{\infty}) = \mathbf{0},$$

where $\{X_t\}$ collects endogenous variables and $\{Z_t\}$ collects exogenous shocks or policy paths.

Sequence Space vs. State Space

Comparison

Intuition

- State space asks: *what is the current state, and how does it transition?*
- Sequence space asks: *given a shock path, what equilibrium outcome path clears all markets?*

Takeaway

When the distribution is the intractable state variable, sequence space replaces a high-dimensional state-space problem with a representation in terms of **equilibrium mappings over aggregate paths**, making analysis and computation much more tractable.

Steady State

Steady State

The Standard Incomplete Markets Model

Core of the canonical HA model: the SIM model

$$V_t(s, a) = \max_{c, a'} u(c) + \beta \mathbb{E} [V_{t+1}(s', a') \mid s], \quad (1)$$

$$\text{s.t. } a' + c = (1 + r_t)a + y_t(s) \quad (2)$$

$$a' \geq a \quad (3)$$

We basically need 4 steps to solve the steady state:

- ➊ Discretize the state space
- ➋ Solve for steady state policy functions $a'(s, a)$ and $c(s, a)$
- ➌ Solve for steady state distribution $D(s, a)$
- ➍ Aggregate assets and consumption

Steady State

Step 1: Discretization

Asset Grid for a :

- It is common to use a uniform grid.
- Most nonlinearity occurs near the borrowing constraint $\underline{a}$, so more points should be placed close to the constraint.
- **Solution:** Use a double exponential grid:

$$a_i = \underline{a} + \exp(\exp(u_i) - 1) - 1$$

where u_i is uniform on $[0, \bar{u}]$.

Markov Chain for s :

- Approximate an AR(1) process with persistence ρ .
- For $\rho \approx 1$, use Rouwenhorst's method.

Steady State

Step 2: Steady State policies

We'll use two optimality conditions:

Envelope Condition:

$$V_{a,t}(s, a) = (1 + r_t) u'(c_t(s, a)) \quad (4)$$

First Order Condition:

$$u'(c_t(s, a)) \geq \beta \mathbb{E} [V_{a,t+1}(s', a'_t(s, a))] \quad (5)$$

with equality unless the budget constraint binds.

The key to solving for steady state policies is to use **EGM** backward iteration. Each iteration proceeds as follows:

$$V_{a,t+1}(s', a') \xrightarrow{(5)} c_t^{\text{endo}}(s, a') \xrightarrow{\text{interpolate}} a'_t(s, a) \xrightarrow{(1)} c_t(s, a) \xrightarrow{(4)} V_{a,t}(s, a)$$

Steady State

Step 3: Distribution

Goal: Update the next period distribution $D_{t+1}(s, a)$ using the policy function $a'_t(s, a)$.

Issue: The policy $a'_t(s, a)$ generally **does not land exactly on the grid points**.

Solution: Use **lotteries** to distribute probability mass. The idea is to map the current mass to adjacent grid points in the next period based on interpolation weights.

Suppose for two grid points a_i, a_{i+1} , the policy yields:

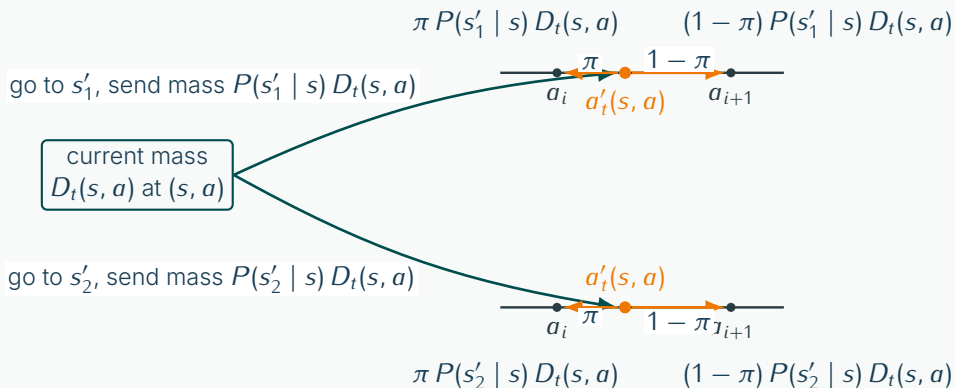
$$a'_t(s, a) = \pi a_i + (1 - \pi) a_{i+1}$$

where $0 \leq \pi \leq 1$. Then, send mass $\pi D_t(s, a)$ to (s', a_i) and $(1 - \pi) D_t(s, a)$ to (s', a_{i+1}) .

Key Idea: No simulation is needed! We iteratively update the distribution until it converges.

Steady State

Step 3: Two-State Lottery Update



First transition across idiosyncratic states, then use the asset lottery within each s' row.

Steady State

Step 4: Aggregation

For each period t , the aggregate variables are easily obtained from the distribution and policy functions:

$$C_t = \sum_{s,a} c_t(s, a) \cdot D_t(s, a)$$

$$A_t = \sum_{s,a} a'_t(s, a) \cdot D_t(s, a)$$

Computing HA Jacobians: the “fake news” algorithm

Computing HA Jacobians: the “fake news” algorithm

Partial Equilibrium Dynamics

Q: Given time-varying $\{r_t\}_{t=0}^T$ or $\{y_t(s)\}_{t=0}^T$, how to get C_t, A_t ?

A: Backward and Forward Iteration!

❶ From $V_{a,T} = V_{a,ss}$, iterate **backward** to get $a'_t(s, a), c_t(s, a)$ for all $t \geq 0$.

❷ From $D_0(s, a) = D_{ss}(s, a)$, iterate **forward** to get $D_t(s, a)$ from $a'_t(s, a)$.

❸ Compute

$$C_t = \sum_{s,a} c_t(s, a) \cdot D_t(s, a), \quad A_t = \sum_{s,a} a'_t(s, a) \cdot D_t(s, a)$$

Computing HA Jacobians: the “fake news” algorithm

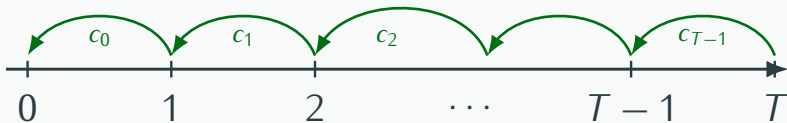
Partial Equilibrium Dynamics



Computing HA Jacobians: the “fake news” algorithm

Partial Equilibrium Dynamics

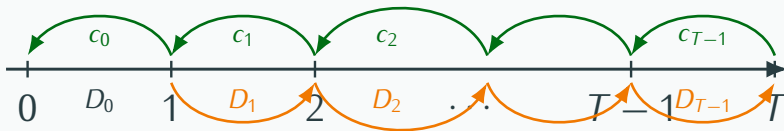
Step 1: Backward to get $c_t(s, a)$



Computing HA Jacobians: the “fake news” algorithm

Partial Equilibrium Dynamics

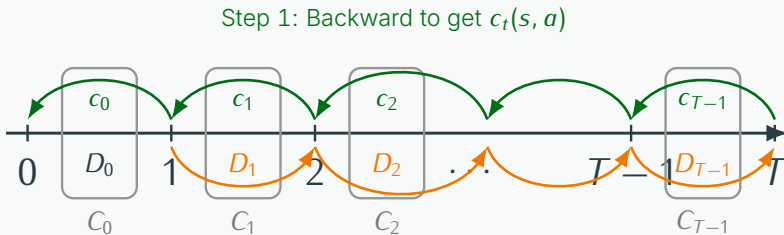
Step 1: Backward to get $c_t(s, a)$



Step 2: Forward to get $D_t(s, a)$

Computing HA Jacobians: the “fake news” algorithm

Partial Equilibrium Dynamics



Step 2: Forward to get $D_t(s, a)$

Step 3: Aggregate to get C_t, A_t

Computing HA Jacobians: the “fake news” algorithm

Sequence-space Jacobians

Denote the impulse response of aggregate consumption C_t to a one-time productivity shock y_s at time s by:

$$\frac{\partial C_t}{\partial y_s}$$

We want the stacked matrix:

$$\mathcal{J}^{C,y} \equiv \left(\frac{\partial C_t}{\partial y_s} \right)_{T \times T} = \left(\mathcal{J}_{ts}^{C,y} \right)_{T \times T}$$

Such a matrix is the **sequence-space Jacobian (SSJ)**.

Computing HA Jacobians: the “fake news” algorithm

How to compute

To compute the SSJ, we truncate to some finite time T (e.g., $T = 300$).

For a generic approach with T columns, one would need T^2 backward iterations and T^2 forward iterations.

Simply loop over all s and t !

Key result: This paper computes the SSJ with only T backward and T forward iterations.

Computing HA Jacobians: the “fake news” algorithm

From $\mathcal{O}(T^2)$ to $\mathcal{O}(T)$: First Row

Denote by $c_t^s(s, a)$ and $a_t^s(s, a)$ the policies at time t in response to a shock at time s .

Key idea 1: Only the time difference $s - t$ until the perturbation matters.

$$c_t^s(s, a) = \begin{cases} c_{ss}(s, a) & \text{if } s < t \\ c_{T-1-(s-t)}^{T-1}(s, a) & \text{if } s \geq t \end{cases}$$

We only compute a single backward iteration with $s = T - 1$ to obtain all the $c_t^s(s, a)$.

For aggregate responses:

- $C_0^s = \sum_{s,a} c_0^s(s, a) D_{ss}(s, a)$, so we get the **first row** of the Jacobian: $\mathcal{J}_{0s}^{C,y} = \frac{\partial C_0}{\partial y_s}$.
- $D_1^s(s, a)$ is then implied by $c_0^s(s, a)$ for all s .

Computing HA Jacobians: the “fake news” algorithm

From $\mathcal{O}(T^2)$ to $\mathcal{O}(T)$: First Column

- Intuitively, we can continue to compute $D_t^s(s, a)$, then aggregate $c_t^s = c_t^{s'} D_t^s$ for all s, t (Still $\mathcal{O}(T^2)$!)

- Observe that to first order,

$$dC_t^s = dc_t^{s'} D_{ss} + c_{ss}' dD_t^s$$

- Consider $s = 0$. For $t \geq 1$, only the second term remains,

$$dC_t^0 = \underbrace{dc_t^{0'} D_{ss}}_{c_t^s(s, a) = c_{ss}(s, a), s < t} + c_{ss}' dD_t^0 = c_{ss}' (\Lambda_{ss}')^{t-1} dD_1^0$$

- Here, Λ_{ss} is the steady-state transition matrix for (s, a) , obtained by $c_{ss}(s, a)$ so

$$dC_t^0 = c_{ss}' (\Lambda_{ss}')^{t-1} dD_1^0 \tag{6}$$

- Then we get the first column of the Jacobian: $\mathcal{J}_{t0}^{C,y} = \frac{\partial c_t}{\partial y_0}$ by (6).

Computing HA Jacobians: the “fake news” algorithm

From $\mathcal{O}(T^2)$ to $\mathcal{O}(T)$: Fake news for the rest!

How about $s > 0$? Given policy symmetry, we have $d\mathbf{c}_t^s = d\mathbf{c}_{t-1}^{s-1}$. It follows:

$$dC_t^s - dC_{t-1}^{s-1} = \cancel{(d\mathbf{c}_t^s - d\mathbf{c}_{t-1}^{s-1})'} D_{ss} + \mathbf{c}_{ss}' (d\mathbf{D}_t^s - d\mathbf{D}_{t-1}^{s-1})$$

Given $\mathbf{c}_t^s = \mathbf{c}_{t-1}^{s-1} \implies \Lambda_t^s = \Lambda_{t-1}^{s-1}$, it is then easy to see

$$D_t^s - D_{t-1}^{s-1} = \prod_{j=1}^{t-1} (\Lambda_j^{s'}) (\Lambda_0^s - I)' D_{ss} = \prod_{j=1}^{t-1} (\Lambda_j^{s'}) (D_1^s - D_{ss})$$

Holding $\Lambda_j^s \equiv \Lambda_{ss}$, we have

$$\mathcal{F}_{ts} dy_s \stackrel{\text{def}}{=} \mathbf{c}_{ss}' (d\mathbf{D}_t^s - d\mathbf{D}_{t-1}^{s-1}) = \mathbf{c}_{ss}' (\Lambda_{ss}')^{t-1} d\mathbf{D}_1^s \equiv \left(\mathcal{J}_{ts}^{C,y} - \mathcal{J}_{t-1,s-1}^{C,y} \right) dy_s \quad (7)$$

We interpret the $\mathcal{F}_{ts}$ as the “fake news” shock on C_t :

- $t = 0$: news shock that period s shock hits $\Rightarrow d\mathbf{D}_1^s$
- $t = 1$: fake news that there won't be a shock at $s \Rightarrow c_{ss} \Rightarrow \Lambda_{ss}$ afterwards

Computing HA Jacobians: the “fake news” algorithm

From $\mathcal{O}(T^2)$ to $\mathcal{O}(T)$: Whole $\mathcal{J}^{C,y}$

Key Idea 2: Recover $\mathcal{J}^{C,y}$ from 1st row, 1st column and $\mathcal{F}$.

News shock is a sequence of fake news shocks!

$$\mathcal{J}^{C,y} = \begin{pmatrix} \mathcal{J}_{00} & \mathcal{J}_{01} & \mathcal{J}_{02} & \dots \\ \mathcal{J}_{10} & \mathcal{J}_{00} + \mathcal{F}_{00} & \mathcal{J}_{01} + \mathcal{F}_{01} & \dots \\ \mathcal{J}_{20} & \mathcal{J}_{10} + \mathcal{F}_{10} & \mathcal{J}_{11} + \mathcal{F}_{11} & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

where, by the recursive structure, representative entries are given by:

$$\mathcal{J}_{ts} = \begin{cases} \mathcal{J}_{t0} & \text{if } s = 0 \\ \mathcal{J}_{0s} & \text{if } t = 0 \\ \mathcal{J}_{t-1,s-1} + \mathcal{F}_{t-1,s-1} & \text{if } t, s \geq 1 \end{cases}$$

Computing HA Jacobians: the “fake news” algorithm

Implementation

Define the expectation function:

$$\mathcal{E}_t(s, a) \equiv \mathbb{E}[c_{ss}(s_t, a_t) \mid s_0 = s, a_0 = a]$$

that is, the expected consumption at time t for an agent starting at (s, a) today.

This is very easy to compute recursively:

$$\mathcal{E}_0 = c_{ss}, \quad \mathcal{E}_t = \Lambda_{ss} \mathcal{E}_{t-1}$$

Key Idea 3: The $\mathcal{E}'_t$ are enough to get all $t \geq 1$ rows of $\mathcal{F}$:

- **First column** ($s = 0$): from (6), $\mathcal{J}_{t0} dy_s = \mathcal{E}'_t dD_1^0$
- **Other columns** ($s > 0$): from (7), $\mathcal{F}_{ts} dy_s = \mathcal{E}'_t dD_1^s$
- **Result:** Eventually only $O(T)$ computation!

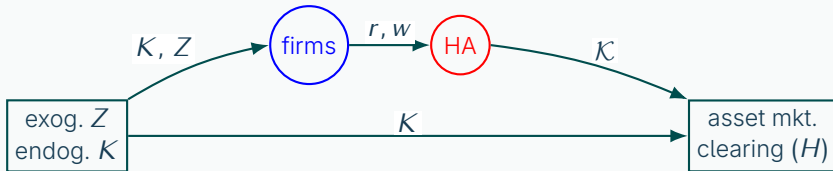
DAG

Krusell-Smith model as a directed acyclic graph (DAG)

Recall that we wrote equilibrium in KS model as system

$$H_t(\mathbf{K}, \mathbf{Z}) \equiv \mathcal{K}_t \left(\left\{ \underbrace{\alpha Z_s K_{s-1}^{\alpha-1} - \delta}_{r_s}, \underbrace{(1-\alpha) Z_s K_{s-1}^{\alpha}}_{w_s} \right\}_{s=0}^{\infty} \right) - K_t = 0$$

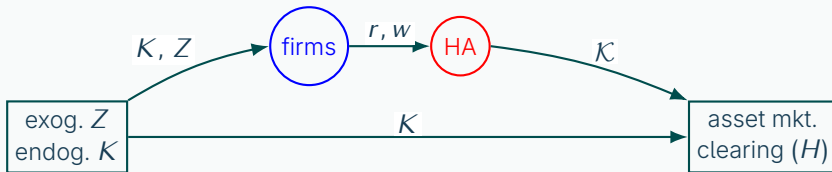
where $\mathcal{K}_t \equiv A_t$ is the aggregate capital stock at time t and $\mathbf{Z} \equiv \{Z_s\}_{s=0}^{\infty}$ is the time-varying TFP process. This corresponds to a simple graph:



DAGs

DAG

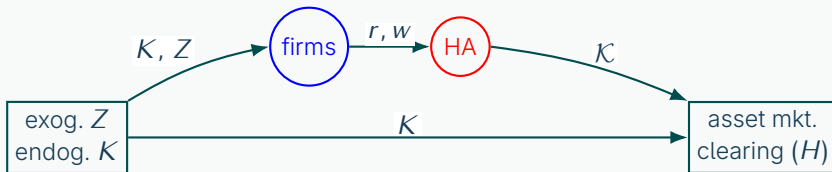
DAGs combining simple and HA blocks



- Each node ("block") takes sequences as inputs and outputs.
- Inputs of later blocks are outputs of earlier blocks.
- The number of endogenous variables ("unknowns") equals the number of equations in H ("targets"); solve to get equilibrium.
- Two kinds of blocks:
 - **Simple blocks:** Analytical equations directly in aggregates, e.g. $r_t = \alpha Z_t K_{t-1}^{\alpha-1} - \delta$; Jacobians are sparse and easy to obtain.
 - **HA blocks:** As described previously.

DAG

DAGs combining simple and HA blocks



- Define $\mathbf{J}^{o,i}$ to be the **total derivative** of o along the DAG with respect to the exogenous or endogenous variable i (distinct from the block Jacobian $\mathcal{J}^{o,i}$).
- Here, for example, $\mathbf{J}^{r,K}$ equals $\mathcal{J}^{r,K}$, etc.: the direct effect is the only effect.
- Then we need the chain rule. For instance:

$$\mathbf{J}^{\mathcal{K},K} = \mathcal{J}^{\mathcal{K},r} \mathbf{J}^{r,K} + \mathcal{J}^{\mathcal{K},w} \mathbf{J}^{w,K}$$

- Finally, we obtain $\mathbf{H}_K$ and $\mathbf{H}_Z$; then $\mathbf{G}^{K,Z} \equiv -\mathbf{H}_K^{-1} \mathbf{H}_Z$ gives the general equilibrium map from Z to K , and all $\mathbf{G}^{o,Z}$ are determined from it.

DAG

General case: solving with forward accumulation

- Initialize $\mathbf{J}^{i,i}$ as the identity for all exogenous or unknown i .
- Evaluate the chain rule following a topological sort:

$$\mathbf{J}^{o,i} = \sum_{m \in I_b} \mathcal{J}^{o,m} \mathbf{J}^{m,i}$$

on blocks b to get $\mathbf{H}_U = \mathbf{J}^{H,U}$ and $\mathbf{H}_Z = \mathbf{J}^{H,Z}$.

- This is called “forward accumulation” in the algorithmic differentiation literature.
- Compute the general equilibrium mapping: $\mathbf{G}^{U,Z} = -\mathbf{H}_U^{-1} \mathbf{H}_Z$
- Then do forward accumulation again:

$$\mathbf{G}^{o,Z} = \sum_{m \in I_b} \mathcal{J}^{o,m} \mathbf{G}^{m,Z}$$

to get general equilibrium mappings $\mathbf{G}^{o,Z}$ from shocks dZ to all variables o of interest.

- This gives impulse responses to every shock simultaneously.

Recap

Recap

- **How to get sequence-space Jacobians?**
 - Fake news algorithm for HA blocks
 - Forward Accumulation on DAG
- **What can we do with them?**
 - Get all impulse responses
 - Second moments, estimation and local determinacy
[not shown here]
 - Compute nonlinear MIT shocks
[not shown here]
- **It's fast, general and accessible!**